

6 artigo "AI test generation: A Dev's Guide without shooting Yourself in the foot" começa destacando o entusiasmo em utilizar ferramentas de IA generativa. Para criar testes automatizados - unitários, de integração e end-to-end - com a promessa de aumentar rapidamente a cobertura de código e economizar esforço manual. No entanto, alerta que pode se enganar: a qualidade do teste depende diretamente da qualidade dos dados com que o modelo foi ~~criado~~ treinado e, se esses dados estiverem repletos de bugs, padrões ~~obsoletos~~ inadequados ou código obsoleto, o resultado pode ser uma bateria de bugs inúteis ou enganosos.

6 texto explica como as LLMs aprendem com enormes repositórios de código, que incluem exemplos corretos e também exemplos cheios de falhas, vulnerabilidades e más práticas. Assim, ao gerar testes, a IA pode tanto gerar código de teste ruim quanto, pior ainda, "validar" comportamentos defeituosos da aplicação, escrevendo como se teste ~~de~~ não-confiáveis. Esse efeito, chamado de "Verification vs Validation Trap", faz o desenvolvedor ter uma falsa segurança.

6 artigo também detalha os principais tipos de problemas que surgem com testes de IA e propõe o uso de prompts de análise estática e especialmente SonarLint como "guardião da qualidade". A ideia é garantir mensagens significativas, repleta de detalhes de menções, testes exatidão, etc... Sem esse crivo automático, os testes podem falhar silenciosamente, "protegendo" bugs.

6 guia reforça que a IA deve ser vista como um auxiliar, mas nunca como substituto do julgamento humano. É imprescindível revisar manualmente cada teste, usar análise estática na IDE e no pipeline de CI, alimentar o modelo com instruções claras e contexto relevante. Só assim será possível aproveitar ganhos de produtividade sem comprometer a qualidade e a confiabilidade de software.